SDLC-STRUCTURED ROADMAP

PesaGuard

Complete Project Lifecycle — Requirements Through Maintenance

Structure	Classic SDLC: Requirements, System Design, Implementation, Testing, Deployment, Maintenance
Consolidates	Master Build Prompt, Gaps Addendum, Tenant/Threat/Billing Addendum, Data Residency/Localization/Scope Addendum
Prepared for	Victor Kipruto Rop · DataForge
Date	July 2026

This document reorganizes every requirement and build phase already specified into a formal SDLC structure, and adds a fully detailed Maintenance stage -- the one phase not yet covered in depth by the prior prompts.

OVERVIEW

The Software Development Life Cycle

A continuous feedback loop runs from Maintenance back into Requirements as real pilot usage surfaces new needs.

PesaGuard's build has, in practice, already moved through most of these stages informally across the six build prompts and roadmap. This document makes that structure explicit and gives the Maintenance stage the same level of detail as the earlier stages -- since a financial reconciliation system's job isn't done at launch, it's done every day after launch.

SDLC Stage	PesaGuard Equivalent	Status
Requirements	Phase 0 discovery + FR/NFR tables (Master Build Prompt)	Spec complete; real validation pending
System Design	Architecture, data models, tech stack (Documentation, Blueprint)	Complete
Implementation	Phases 1-14 (Master Build Prompt + 3 addenda)	Specified; not yet built beyond MVP scaffold
Testing	Phase 10 testing & hardening requirements	Specified
Deployment	Phase 11 deployment & launch	Specified
Maintenance	Not previously detailed as its own stage	New in this document

01 Requirements Gathering & Analysis

Corresponds to Phase 0 of the Master Build Prompt plus the full functional and non-functional requirements tables already specified. This stage's output is a validated problem statement and a complete requirements set -- not code.

Activities

- Identify and interview 5-10 candidate pilot customers (SACCOs, e-commerce operators)
- Validate that manual reconciliation pain is real and access to internal systems is grantable
- Confirm Daraja sandbox access and collect real callback payload samples
- Document functional requirements (20 items, FR-1 through FR-20) and non-functional requirements (7 items, NFR-1 through NFR-7) as specified in the Master Build Prompt
- Resolve open product questions directly with candidates: Kiswahili vs. English, data residency preference, payment methods beyond M-Pesa

Exit Criteria

- At least one real or realistic pilot candidate has validated the core problem
- Requirements tables are confirmed against real conversations, not assumptions alone

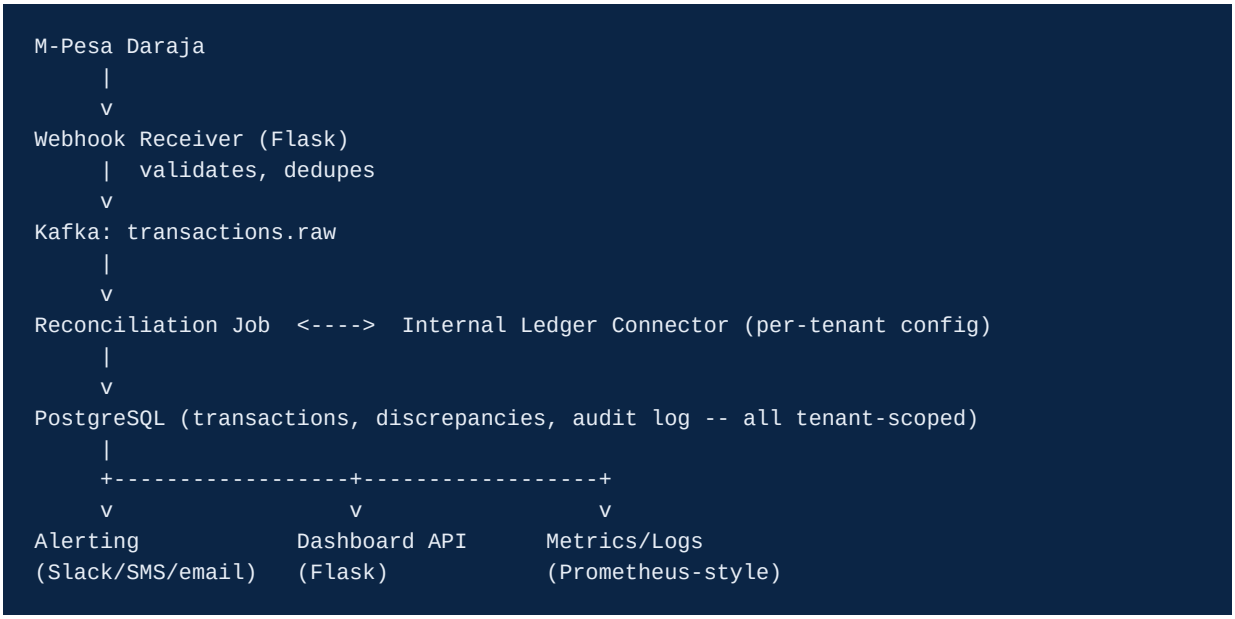
Gate

Do not proceed to Implementation until this stage's exit criteria are met. System Design work below can proceed in parallel since it doesn't depend on customer-specific answers.

02 System Design

The architecture, data models, and technology decisions that Implementation builds against.

High-Level Architecture



Design Principles

- Kafka decouples ingestion from processing so a slow rule engine or a down database never causes a Daraja webhook timeout
- Every table, Kafka message, and API route carries tenant_id from the first migration
- Postgres is the single source of truth for audit purposes
- Connectors are pluggable behind a BaseConnector interface

Tech Stack

Layer	Technology
Ingestion	Flask (Python)
Streaming	Apache Kafka
Processing	Python consumer; PyFlink migration path documented, not built until volume requires it
Primary storage	PostgreSQL
Dedup / caching	Redis
Connectors	psycopg2, gspread (OAuth2), requests
Alerting	Slack Incoming Webhooks, Africa's Talking (SMS)
Dashboard frontend	Next.js + React

Layer	Technology
Dashboard backend	Flask or Django REST Framework
Monitoring	Prometheus + Grafana
Containerization	Docker, docker-compose (single-VM production default)
CI/CD	GitHub Actions

Data Model Summary

Entity	Key Fields	Notes
Transaction	trans_id, trans_amount, msisdn, tenant_id, raw_payload	One row per M-Pesa event
Discrepancy	id, trans_id, anomaly_type, resolved, tenant_id	One row per rule violation
InternalRecord	internal_ref, amount, phone_number, status, tenant_id	Synced from tenant's ledger
Tenant	tenant_id, status, shadow_mode, thresholds	Provisioning and config state
RuleVersion	version_id, tenant_id, thresholds, effective_date	Ties discrepancies to rules active at the time

03 Implementation

The 15 build phases from the Master Build Prompt and its three addenda, condensed here in build order.

Phase	Focus	Source
1	Core pipeline: ingestion, Kafka, tenant-scoped Postgres, basic anomaly rules, audit log	Master Build Prompt
2	Real matching logic: connectors, phone+amount+time-window matching, needs_review tier	Master Build Prompt
3	Alerting system: multi-channel, severity tiers, escalation, digest	Master Build Prompt
4	Monitoring: /metrics, structured logs, operator alerting on a separate channel	Master Build Prompt
5	Dashboards: Grafana (operational) + Next.js (customer-facing)	Master Build Prompt
6	Security & API protection: rate limiting, source validation, tenant isolation	Master Build Prompt
7	Backup, DR & data retention: tested restores, offboarding process	Master Build Prompt
8	Incident response tooling: paging, severity definitions, postmortems	Master Build Prompt
9	API documentation: OpenAPI spec, interactive docs	Master Build Prompt
12	Operational gaps: OAuth token mgmt, staging, dependency scanning, RBAC, export, versioning, rotation	Gaps Addendum
13	Tenant lifecycle, threat modeling, feature flags, billing, help docs	Tenant/Threat/Billing Addendum
14	Data residency, localization, payment-method scope decisions	Addendum III

Phases 10 (Testing) and 11 (Deployment) are treated as their own SDLC stages below.

Implementation Constraints (apply throughout)

- Don't introduce Kubernetes, Terraform, BigQuery, or a paid observability SaaS unless a stated scale reason exists
- Every new module gets a docstring explaining the business problem it solves
- Flag assumptions about Daraja behavior, thresholds, or retention periods explicitly rather than treating a guess as final

04 Testing

Comprehensive verification before anything reaches a real customer's data.

Test Type	What It Verifies
Unit tests	Every anomaly rule and matching scenario (exact, partial, no-match, late-match, duplicate)
Integration tests	Full pipeline against docker-compose with realistic Daraja sandbox fixtures
Idempotency test	Duplicate TransID delivered twice produces exactly one processed record and one alert
Load test	Burst traffic produces zero dropped events and zero dropped alert deliveries
Multi-tenant isolation test	Tenant A's queries/API calls never surface tenant B's data, even adversarially
Escalation timing test	Unresolved critical alerts re-fire on schedule, stop after acknowledgement
Channel separation test	Operator alerts and customer alerts never cross-contaminate channels
RBAC enforcement test	A viewer role cannot hit admin-only routes even via direct API call
Webhook spoofing test	A spoofed payload is rejected by source validation
Billing isolation test	A tenant never sees another tenant's invoice or usage data

Exit Criteria

- All test categories above pass in CI on every merge to main
- No known critical-severity gap remains undocumented in THREAT_MODEL.md

05 Deployment

Getting the tested system safely in front of a real pilot tenant.

Deployment Requirements

- Single-VM production deployment via docker-compose (default; Kubernetes only if explicitly justified by scale)
- GitHub Actions CI: test, build, deploy on merge to main
- Secrets injected via environment/secrets manager, never committed
- HTTPS on the webhook receiver (required by Daraja for the ConfirmationURL)
- Staging environment (Daraja sandbox, isolated data) used for pre-production validation

Tenant Onboarding & Launch Sequence

- Provision the tenant with shadow_mode = true by default
- Connector setup and validation against the tenant's real internal system
- Run in shadow mode (discrepancies logged, alerts suppressed) for at least one week
- Review shadow-mode results with the tenant before flipping to active alerting
- Only then does the tenant move to active status with real customer-facing alerts

06 Maintenance & Operations

The stage prior documents treated as scattered concerns (backups, rotation, incident response) rather than a unified ongoing practice. This is what running PesaGuard looks like every week after launch, indefinitely.

Daily / Continuous

- Monitor Grafana dashboards: Kafka lag, reconciliation latency, discrepancy rate, alert delivery success
- Review any operator alerts routed to the ops channel
- Confirm daily automated Postgres backup completed successfully

Weekly

- Review open discrepancies across all tenants for patterns suggesting a rule needs tuning
- Check dependency vulnerability scan results and triage any new findings
- Review any tenant in shadow_mode approaching the end of their observation period

Monthly

- Review resolution-note data from the dashboard for threshold-tuning signal
- Spot-check the disaster recovery restore procedure against a scratch environment
- Review per-tenant connector health and proactively reach out if sync failures are rising
- Review billing/usage data for accuracy before invoicing

Quarterly

- Full disaster recovery restore drill against a scratch environment
- Revisit THREAT_MODEL.md, especially if any new connector type or alert channel shipped
- Review and rotate secrets per the documented rotation schedule
- Review retention policy compliance: confirm data is actually deleted/archived on schedule

Ongoing / Event-Driven

- Incident response: follow INCIDENT_RESPONSE.md whenever a real incident occurs
- Rule rollout: follow RULE_ROLLOUT.md when tuning a threshold
- Tenant offboarding: follow the documented export + delete process with its grace period
- Apply security patches promptly; treat CI failures on dependency bumps as a signal to investigate, not to pin and ignore

Maintenance Ownership Table

Activity	Frequency	Reference Document
Dashboard/metrics review	Daily	RUNBOOK.md
Backup verification	Daily	DISASTER_RECOVERY.md
Dependency scan triage	Weekly	SECURITY.md
Threshold/rule review	Weekly-Monthly	RULE_ROLLOUT.md
Connector health review	Monthly	RUNBOOK.md

Activity	Frequency	Reference Document
DR restore drill	Quarterly (full) / Monthly (spot-check)	DISASTER_RECOVERY.md
Threat model revisit	Quarterly or on new feature	THREAT_MODEL.md
Secrets rotation	Quarterly	SECRETS_ROTATION.md
Retention compliance check	Quarterly	DATA_RETENTION.md
Incident response	As needed	INCIDENT_RESPONSE.md
Tenant offboarding	As needed	DATA_RETENTION.md

07 The Feedback Loop Back to Requirements

SDLC models are drawn as a line, but PesaGuard's actual lifecycle is a loop. Maintenance-stage findings should feed back into Requirements on a regular cadence, not just when something breaks.

Maintenance Signal	Feeds Back Into
Resolution-note patterns from the dashboard	Requirements: tuning matching thresholds (Phase 2 / RULE_ROLLOUT.md)
Recurring connector failures for a tenant type	System Design: whether the connector abstraction needs to evolve
Alert fatigue reports from a tenant	Requirements: severity tier definitions (Phase 3)
New payment method requests from real tenants	Requirements: revisiting the payment-method scope decision (Phase 14)
Repeated support questions	Requirements: end-user help documentation gaps (Phase 13)

This closes the lifecycle. Requirements through Maintenance are now fully specified as one connected system. The next action outside this document remains the same as before: validating Requirements with real conversations before Implementation goes further than the current MVP scaffold.